

3. Objetos y prototipos

3.1. Objetos: principios básicos

Los objetos son el tipo de datos complejo disponible en JavaScript. Un objeto está formado por **colecciones** de datos definidas como **propiedades**.

! IMPORTANTE

Es muy importante tener claro que JavaScript tiene objetos, pero **no tiene clases**, por lo que es diferente de otros lenguajes como Java. Sí que existe la posibilidad de definir objetos mediante la palabra reservada **class**, pero no tiene nada que ver con las clases de lenguajes como Java.

La manera más sencilla de crear un objeto en JavaScript es mediante un **literal**:

```
let coche = {
  color: "rojo",
  marca: "seat",
  arrancar: function() {
    console.log("Arrancando");
  }
}

console.log( coche.color ); // rojo
console.log( coche.marca ); // seat
coche.arrancar(); // Arrancando
```

Se pueden añadir propiedades a un objeto directamente. Existen dos modos de hacerlo:

```
coche.modelo = "ibiza";
coche["combustible"] = "diesel";
```

Se pueden iterar las propiedades de un objeto con el operador **for in**. Así, en el ejemplo anterior:

```
for (let prop in coche) {
  console.log(prop);
}

// color
// marca
// arrancar
```

Si se accede a una propiedad de un objeto que no existe, el código **no genera error**, sino que devuelve **undefined**. Es posible detectar si una propiedad está definida en un objeto mediante el operador **in**:

```
let persona = {
  edad: 20,
  nombre: "David"
}

console.log( "edad" in persona ); // true
console.log( "apellido" in persona ); // false
```

3.2. Asignación desestructurante

En ocasiones, nos encontramos con que queremos **extraer** determinadas propiedades de un objeto o elementos de un *array* y asignarlas a variables independientes. Para ello se puede utilizar la **asignación desestructurante**.

En *arrays*, la asignación se realiza por **posición** y se utilizan **corchetes** `[ ]`. Es posible también utilizar el operador *rest* `...` para obtener el resto de parámetros en otro *array*.

EJEMPLO

En la documentación complementaria de la unidad, situada en la sección *Material de trabajo / Descarga de documentos*, dispones del archivo `DWEC_U02_A03_01.js`, con el código de ejemplo.

En el caso de utilizar **objetos**, la asignación se realizará por **nombre** y mediante **llaves** `{ }`, y es posible también utilizar el operador *rest* `...`, que en este caso almacenará el resto de parámetros en un **objeto**.

IMPORTANTE

JavaScript considera las llaves `{ }` como delimitadores de bloque. Por ello, si se utiliza una asignación desestructurante con variables previamente definidas, esto es, del modo `{ /* Variables */ } = OBJETO`, **se generará un error**, ya que se considera un bloque de código y no una asignación. Si se presenta el caso, se debe escribir la asignación **entre paréntesis**.

Si la asignación incluye también la declaración, del modo `let { /* Variables */ } = OBJETO`, no habrá problema.

EJEMPLO

En la documentación complementaria de la unidad, situada en la sección *Material de trabajo / Descarga de documentos*, dispones del archivo `DWEC_U02_A03_02.js`, con el código de ejemplo.

3.3. Referencia y copia

Las variables **no almacenan objetos**, sino que **guardan referencias a ellos**. Así:

- Un objeto que no esté referenciado por ninguna variable **se elimina** de memoria.
- **No es posible copiar un objeto** realizando una **asignación** a otra variable: seguirá existiendo el mismo objeto referenciado por dos variables.
- Para hacer una **copia** de un objeto, se puede utilizar `Object.assign(OBJETO_DESTINO, OBJETO_ORIGEN)`.

EJEMPLO

En la documentación complementaria de la unidad, situada en la sección *Material de trabajo / Descarga de documentos*, dispones del archivo `DWEC_U02_A03_03.js`, con el código de ejemplo.

3.4. Objetos predefinidos y métodos de primitivas

Los tipos de datos **primitivos** pueden convertirse a **objetos**. Tal como veremos más adelante, JavaScript define un mecanismo de **herencia** basada en **prototipos**, de tal manera que los objetos pueden **acceder a propiedades** definidas en sus prototipos.

Esto tiene una serie de ventajas a la hora de tratar con determinados tipos de datos. Por ejemplo, al tratar con *strings* nos puede interesar calcular su longitud o convertir a minúsculas; al tratar con números nos puede interesar convertirlos a notación científica.

Para ello, JavaScript incorpora una serie de **objetos predefinidos en el lenguaje**, que actúan como prototipos que incluyen las **funcionalidades más comunes**. Entre ellos están los objetos **String** o **Number** (observa que tienen una mayúscula inicial, lo que los identifica como objetos y no como tipos primitivos), que se utilizan para tratar con sus tipos de datos asociados (**string** y **number**). JavaScript incluye también el mecanismo para **convertir los tipos de datos primitivos a objetos complejos** que tengan acceso a esas funcionalidades. En el caso de los *strings*, la conversión se realiza de manera **automática**, mientras que en otros casos, como los **tipos de datos numéricos**, es necesario hacerlo de manera **explícita**. Una vez hecha la conversión para utilizar la funcionalidad correspondiente, el dato **se deja de nuevo como tipo de dato primitivo**.

! IMPORTANTE

Esta es la razón por la que podemos ejecutar un código como el siguiente:

```
let cadena = "Texto";  
console.log( cadena.length ); // 5
```

cadena almacena un tipo de datos primitivo, no un objeto; pero, al ejecutar **cadena.length**, JavaScript la convierte temporalmente a un **objeto** de tipo **String**, que tiene acceso mediante herencia a la propiedad **length**, que devuelve la longitud de la cadena.

A continuación, puedes observar algunos ejemplos de métodos de primitivas:

```
Number(2.54).toExponential(); // '2.54e+0'  
"texto".toUpperCase(); // "TEXTO"  
Number(25).toString(); // "25"
```

```
let cadena = "Esto es un texto";  
cadena.indexOf("x"); // 13  
cadena.includes("texto"); // true  
cadena.startsWith("Es"); // true  
cadena.endsWith("Es"); // false
```

3.5. Métodos de *arrays*

Los *arrays* en JavaScript son **objetos**. Además, tienen acceso por herencia a una serie de funciones muy útiles.

EJEMPLO

En la documentación complementaria de la unidad, situada en la sección *Material de trabajo / Descarga de documentos*, dispones del archivo **DWEC_U02_A03_04.js**, con el código de ejemplo.

3.6. This

this en JavaScript es una palabra reservada que se utiliza dentro de los métodos de los objetos para hacer referencia al objeto sobre el que se llama a dicho método en el lugar de ejecución, no en la definición. El uso de **this** con frecuencia causa confusión, ya que su comportamiento es distinto del de otros lenguajes de programación. Consideremos el siguiente código:

```
let usuario = {
  nombre: "David",
  saluda: function() { // Lugar de definición de "saluda"
    return this.nombre;
  }
}

// Lugar de ejecución de "saluda"
// Se llama a la función sobre el objeto
usuario.saluda(); // David

// Lugar de ejecución de "saluda"
// Se llama a la función sin referencia al objeto
let intentaSaludar = usuario.saluda;
intentaSaludar(); // undefined
```

Como puede observarse, en apariencia, ambas ejecuciones de la función deberían producir el mismo resultado, dado que, en la definición, la función **saluda** hace referencia a **this**, que parece apuntar al objeto **usuario**. Sin embargo, vemos que solo funciona la primera versión, **usuario.saluda()**, mientras que la segunda, **intentaSaludar()**, pese a ser una variable que apunta a la misma función, no. La llamada **usuario.saluda()** realiza un enlace implícito (*implicit binding*).

Existen otras maneras de asociar la ejecución del método a un objeto, aparte del enlace implícito. Una posibilidad es utilizar el enlace explícito (*explicit binding*). Para ello, se utilizan las funciones **call**, **apply** o **bind**. Estas funciones están definidas en el prototipo de cualquier función, por lo que cualquier función puede hacer uso de ellas.

EJEMPLO

En la documentación complementaria de la unidad, situada en la sección *Material de trabajo / Descarga de documentos*, dispones del archivo **DWEC_U02_A03_05.js**, con el código de ejemplo.

Por último, es necesario volver a señalar que las funciones flecha no definen su propio **this**:

```
let usuario = {
  nombre: "David",
  saluda: () => { // Función 'saluda' definida como función flecha
    return this.nombre;
  }
}

// Lugar de ejecución de "saluda"
// Se llama a la función sobre el objeto
// 'this' no está definido, por ser una función flecha
usuario.saluda(); // undefined
```


¿Por qué esta diferencia? En ocasiones, el uso de **this** puede no funcionar como nosotros esperamos, sobre todo en el uso de *callbacks*. Consideremos el siguiente ejemplo con la función **setTimeout**, que es una función que pone en marcha un temporizador que ejecuta una función al terminar:

```
let usuario = {
  nombre: "David",
  saludaConRetardo: function() {
    setTimeout(function() {
      // Intenta mostrar el nombre del usuario
      console.log(this.nombre);
    }, 1000); // Retardo de 1000ms (1s)
  }
}

usuario.saludaConRetardo(); // undefined
```

Pese a llamar a la función haciendo referencia al objeto mediante **usuario.saludaConRetardo()**, la función no escribe en consola el nombre. ¿Por qué? Porque **setTimeout** es una función asíncrona que espera un tiempo y luego ejecuta la función pasada en su primer parámetro. Dicha llamada **se ejecuta sin referencia al objeto**, por lo que **this** no apunta al objeto que esperamos, sino que apunta al objeto global (**window**, en el navegador).

Para conseguir la funcionalidad deseada, podemos utilizar un **enlace explícito** (con **bind**, por ejemplo). Para ello debemos conseguir que la función **setTimeout** se ejecute sobre el objeto que queremos. Como la llamada a dicha función se realiza dentro del método **saludaConRetardo**, **this** es la referencia a dicho objeto. El código quedaría así:

```
let usuario = {
  nombre: "David",
  saludaConRetardo: function() {
    setTimeout(function() {
      console.log(this.nombre);
    }).bind(this), 1000);
    // Mediante 'bind(this)', conseguimos que la función
    // 'setTimeout' tenga como contexto el mismo objeto
    // que la función 'saludaConRetardo'

  }
}

// 'saludaConRetardo' tiene como contexto 'usuario' por enlace implícito en
su ejecución
// Como hemos utilizado 'bind', 'setTimeout' también tendrá el mismo
contexto.
// Por tanto, 'this.nombre' apuntará a 'usuario.nombre', tal como queremos
usuario.saludaConRetardo(); // David
```

Como puedes comprobar, el código anterior puede ser algo complejo de entender. Como alternativa, se puede utilizar una función flecha. El código quedaría así:

```
let usuario = {
  nombre: "David",
  saludaConRetardo: function() {
    setTimeout( () => {
      // Al utilizar una función flecha, 'setTimeout'
      // no define su propio 'this'. JavaScript busca entonces
      // 'this' en el ámbito exterior (ámbito léxico), es decir,
      // el cuerpo de la función 'saludaConRetardo',
      // que es justo lo que queremos
      console.log(this.nombre);
    }, 1000);
  }
}

usuario.saludaConRetardo(); // David
```

Como puedes comprobar, el código con la función flecha es más fácil e intuitivo. La función flecha está definida en el lugar donde no queremos **this**, es decir, la función de **setTimeout**; la función **saludaConRetardo** sí que debe estar definida como función convencional.

! IMPORTANTE

Las reglas y el orden para determinar qué es **this** son las siguientes:

1. Si se utiliza el operador **new** (lo veremos más adelante), es el objeto devuelto por el constructor.
2. Si se utiliza enlace explícito [**call**, **apply** o **bind**], es el objeto especificado.
3. Si se utiliza enlace implícito, es el objeto especificado.
4. Si no se aplica nada de lo anterior, es el objeto global o **undefined** en modo estricto.

3.7. Prototipos y herencia

Como hemos comentado anteriormente, JavaScript es un lenguaje orientado a objetos que utiliza un sistema de herencia basada en prototipos.

! IMPORTANTE

En JavaScript **no hay clases**: hay **objetos enlazados a otros objetos**, denominados **prototipos**. Las propiedades de los prototipos **no se heredan**, es decir, no se copian a los objetos nuevos. Los objetos acceden a ellas a través de un mecanismo denominado **delegación**. Al tratar de acceder a una propiedad (incluyendo funciones) de un objeto, JavaScript busca primero si está definida en el objeto; si no lo está, busca en su prototipo.

Al crear un objeto, salvo casos especiales en que así lo establezcamos, se define una propiedad oculta que apunta a un objeto asociado: ese objeto es su **prototipo**. Cuando se intenta acceder a una **propiedad** de un objeto, JavaScript busca en primer lugar si está definida en dicho objeto; si no lo está, **busca en su prototipo**, y así sucesivamente en toda la cadena de dependencias, hasta llegar al objeto base (que suele ser **Object.prototype**). El siguiente código así lo demuestra:

```
let usuario = {
  nombre: "David"
}

// El método 'toLocaleString' funciona, pese a no haberlo definido
// Esta función está definida en 'Object.prototype'
usuario.toLocaleString(); // '[object Object]'
```

Este mecanismo nos permite acceder a una **jerarquía de objetos enlazados a otros objetos**. Ya hemos hablado de alguna de ellas, como la que nos permite trabajar con las funciones específicas de los *arrays*. Además de la jerarquía de objetos disponibles en el lenguaje, **podemos definir nuestra propia estructura de objetos enlazados mediante herencia prototípica**. Ahora bien, al tratarse de un tema complejo, no lo estudiaremos en profundidad.

Los mecanismos más utilizados para construir objetos basados en prototipos son dos: las **funciones constructoras** y los **objetos enlazados**. Existe también una tercera opción, la sintaxis **class**, que veremos en un epígrafe posterior.

A. FUNCIONES CONSTRUCTORAS

Una función constructora es una **función convencional** que define una serie de **propiedades y métodos**. Para hacer **referencia** a las propiedades y a otros métodos, hace uso del operador **this**. Y para crear objetos basados en ella utiliza el operador **new**.

EJEMPLO

En la documentación complementaria de la unidad, situada en la sección *Material de trabajo / Descarga de documentos*, dispones del archivo **DWEC_U02_A03_06.js**, con el código de ejemplo.

La función constructora define las propiedades que tendrán los objetos creados a partir de ella. Además, creará un enlace de dichos objetos a **NOMBRE_FUNCION.prototype**, que será el prototipo al que están enlazadas. Si se crea una **nueva propiedad** (incluyendo funciones) en el **prototipo**, los objetos **podrán acceder a ella**. Si se **modifica alguna propiedad definida en el prototipo**, se **copia al objeto** y no se modifica la del prototipo.

IMPORTANTE

El mecanismo de lectura y modificación de propiedades de los objetos es más complejo de lo que hemos visto. Si quieres más información, puedes buscar información sobre *getters* y *setters*.

B. OBJETOS ENLAZADOS

La segunda técnica para crear objetos basados en prototipos se basa en la función **Object.create** y utiliza exclusivamente objetos, no funciones constructoras. Por tanto, no utiliza tampoco el operador **new**.

EJEMPLO

En la documentación complementaria de la unidad, situada en la sección *Material de trabajo / Descarga de documentos*, dispones del archivo **DWEC_U02_A03_07.js**, con el código de ejemplo.

Este método es **más simple y esclarecedor** que el anterior, ya que no utiliza **new** y no hace referencia a **prototipe**. Como contrapartida, es menos común, sobre todo en código antiguo.

3.8. Prototipos nativos

JavaScript define un conjunto de prototipos incluidos en el lenguaje. Los más importantes son:

- **Object.prototype**. Es el **prototipo base**. Todos los objetos están enlazados con él (a no ser que se construya un objeto sin prototipo, que también es posible). Define algunos métodos básicos, como **toString()**, que convierte cualquier objeto a una cadena.
- **Function.prototype**. Es el prototipo con el que están enlazadas todas las **funciones**. Define, entre otros, los métodos **call**, **apply** y **bind**.
- Prototipos de primitivas. Ya hemos hablado de algunos de ellos, como **String** y **Number**.
- **Array.prototype**. Es el prototipo con el que están enlazados los **arrays**. Define varios métodos y propiedades muy útiles, como **length**, **pop**, **push**, **splice**, **concat**, etcétera.
- **Date.prototype**. Es el prototipo con el que se construyen los objetos de tipo **Date**. Define todos los métodos y propiedades relacionados con los objetos de fecha, como **getDate**, **getMinutes**, **getHours**, etcétera.

! IMPORTANTE

Puedes ver las propiedades definidas en los prototipos tanto en la consola de Node como en la del navegador. Para ello, escribe el nombre del prototipo (por ejemplo, **Date.prototype**) en la consola correspondiente. A continuación, verás la lista de métodos si estás en el navegador web; si estás en NodeJS podrás verlos pulsando la tecla **tabulador**.

3.9. Clases

JavaScript ha incorporado una **nueva sintaxis** para trabajar con objetos. Dicha sintaxis está creada para facilitar la tarea de las personas que vengan del mundo de Java u otro lenguaje orientado a objetos que utilice clases.

! IMPORTANTE

De nuevo, es importante recordar que en JavaScript no existen las clases en el sentido «clásico» (Java, C++). Solo existen objetos relacionados mediante prototipos.

A continuación, se muestra un ejemplo de definición de objetos mediante clases.

```
// Definición de clase
class Usuario {
  constructor(nombre, apellidos) {
    this.nombre = nombre;
    this.apellidos = apellidos;
  }

  // No hay coma entre las definiciones de los métodos

  saludar() {
    console.log(`Me llamo ${this.nombre} ${this.apellidos}`);
  }
}

// Creación de un objeto basado en la clase
// (En realidad, es un objeto cuyo prototipo es el objeto Usuario)
let usuario = new Usuario("David", "Pérez");
usuario.saludar(); // Me llamo David Pérez
```


También es posible crear objetos enlazados a otros mediante herencia prototípica. Para ello se utiliza **extends**, en línea con otros lenguajes que utilizan clases. De esta manera, da la impresión de que existe herencia clásica, aunque en realidad lo que existe es una **delegación**, tal como hemos comentado en apartados anteriores.

EJEMPLO

En la documentación complementaria de la unidad, situada en la sección *Material de trabajo / Descarga de documentos*, dispones del archivo **DWEC_U02_A03_08.js**, con el código de ejemplo.

La sintaxis de clases también utiliza la palabra reservada **super** para **acceder a las propiedades del prototipo enlazado**. También se utiliza para **modificar los constructores** y **llamar al constructor del prototipo**.

EJEMPLO

En la documentación complementaria de la unidad, situada en la sección *Material de trabajo / Descarga de documentos*, dispones del archivo **DWEC_U02_A03_09.js**, con el código de ejemplo.

IMPORTANTE

Recuerda: si extiendes una clase y sobrescribes su constructor, debes llamar al constructor del prototipo mediante **super**.

3.10. JSON

JSON, *JavaScript object notation*, es un formato **basado en texto** para representar objetos. Dada su importancia para enviar y recibir datos entre clientes y servidores, JavaScript define los métodos:

- **JSON.stringify**. Para convertir objetos a JSON.
- **JSON.parse**. Para convertir JSON a objetos JavaScript.

```
let usuario = {
  nombre: "David",
  edad: 25,
  permisos: ["administrador", "editor"]
}

let cadenaJson = JSON.stringify(usuario);
// cadenaJson es un string

let usuarioRecuperado = JSON.parse(cadenaJson);
// usuarioRecuperado es un objeto
```